Snowflake w praktyce

Pipeline zamówień sklepu internetowego: Stage, COPY, INSERT INTO, Stream, Task i Time Travel

Warsztat praktyczny · 4 moduły z weryfikacją

Poziom: średni

Spis treści

1. Scenariusz i plan warsztatu ... 3
1.1. Scenariusz biznesowy ... 3
1.2. Architektura rozwiązania ... 3
1.3. Agenda ... 3
1.4. Wymagania ... 3
1.5. Oczekiwane liczby ... 4
2. Przygotowanie środowiska (start od zera) ... 5
3. Moduł 1. Stage i COPY INTO – ładowanie plików ... 8
Weryfikacja modułu 1 ... 9
4. Moduł 2. Stream i INSERT INTO – przetwarzanie przyrostowe ... 11
Weryfikacja modułu 2 ... 12
5. Moduł 3. Task – automatyzacja pipeline ... 13
Weryfikacja modułu 3 ... 14
6. Moduł 4. Time Travel – odtwarzanie po awarii ... 16
Weryfikacja modułu 4 ... 17
7. Podsumowanie i sprzątanie ... 18
7.1. Co zbudowaliśmy ... 18
7.2. Pytania kontrolne ... 18
7.3. Sprzątanie ... 18

1. Scenariusz i plan warsztatu

1.1. Scenariusz biznesowy

Firma **ShopFlow** prowadzi sklep internetowy z akcesoriami komputerowymi i sprzedaje do Polski, Niemiec, Czech, Słowacji i Litwy. System sprzedażowy co noc eksportuje zamówienia z poprzedniego dnia do pliku CSV. Pliki trafiają do strefy wymiany danych (stage) w Snowflake. Zespół danych ma zbudować pipeline, który:

- ładuje pliki CSV z zamówieniami do warstwy surowej (`RAW`),
- wykrywa tylko nowe wiersze i przenosi je do warstwy analitycznej (`ANALYTICS`), wzbogacając je o dane produktów i wartość zamówienia,
- działa automatycznie, bez ręcznego uruchamiania,
- pozwala szybko cofnąć skutki błędu człowieka (np. UPDATE bez WHERE albo przypadkowy DROP TABLE).

Każdy moduł kończy się zapytaniem weryfikacyjnym, które zwraca listę testów ze statusem **PASS** lub **FAIL**.

1.2. Architektura rozwiązania

Warstwa	Obiekt	Rola w pipeline
Źródło	`@RAW.LANDING_STAGE/orders/`	Wewnętrzny stage z plikami CSV z zamówieniami
Ładowanie	`COPY INTO RAW.ORDERS_RAW`	Ładuje pliki ze stage do tabeli surowej, pomija pliki już załadowane
Wykrywanie zmian	`RAW.ORDERS_RAW_STREAM`	Stream (append-only) śledzący nowe wiersze w tabeli surowej
Transformacja	`INSERT INTO ANALYTICS.ORDERS_CLEAN`	Wzbogacenie o produkt, kategorię, wartość netto
Automatyzacja	`PIPELINE.TSK_LOAD_ORDERS` → `PIPELINE.TSK_TRANSFORM_ORDERS`	Graf zadań: najpierw COPY, potem INSERT ze streamu
Ochrona danych	Time Travel, `CLONE … BEFORE`, `UNDROP`	Odtwarzanie danych po błędach

Przepływ danych: **Pliki CSV → Stage → COPY INTO → ORDERS_RAW → Stream → Task (INSERT INTO) → ORDERS_CLEAN**, a nad wszystkim Time Travel jako siatka bezpieczeństwa.

1.3. Agenda

Część	Weryfikacja
Wprowadzenie i przygotowanie środowiska (skrypt startowy)	Test startowy
Moduł 1. Stage i COPY INTO – ładowanie plików	5 testów
Moduł 2. Stream i INSERT INTO – przetwarzanie przyrostowe	5 testów
Moduł 3. Task – automatyzacja pipeline	5 testów
Moduł 4. Time Travel – odtwarzanie po awarii	5 testów
Podsumowanie i sprzątanie	–

1.4. Wymagania

- Konto Snowflake

- Rola **ACCOUNTADMIN** do jednorazowego nadania uprawnienia `EXECUTE TASK`; resztę warsztatu wykonujemy jako **SYSADMIN**.
- Nie są potrzebne żadne pliki zewnętrzne – wszystkie dane generuje skrypt startowy bezpośrednio w Snowflake.

1.5. Oczekiwane liczby

Moment	ORDERS_RAW	ORDERS_CLEAN
Po module 1 (pliki 01.11, 02.11 i plik late)	758 (400 + 350 + 8)	– (tabela jeszcze nie istnieje)
Po module 2 (+2 zamówienia ręczne)	760	760
Po module 3 (+ plik 03.11, 250 wierszy)	1010	1010
Po module 4 (odtworzenie po awarii)	1010	1010

2. Przygotowanie środowiska (start od zera)

Skrypt tworzy warehouse, bazę, schematy, formaty plików, stage, tabelę produktów oraz generuje w Snowflake 1000 syntetycznych zamówień z trzech dni. Następnie wyładowuje (unload) do stage pliki CSV dla dwóch pierwszych dni oraz jeden plik z błędami. Plik z trzeciego dnia „dotrze" dopiero w module 3.

Skopiuj cały skrypt do nowego arkusza w Snowsight i uruchom go w całości (**Run All**, Ctrl+Shift+Enter).

Krok 0.1. Rola, warehouse, baza i schematy

```
-- Jednorazowo: uprawnienie do uruchamiania tasków dla roli SYSADMIN
USE ROLE ACCOUNTADMIN;
GRANT EXECUTE TASK ON ACCOUNT TO ROLE SYSADMIN;

USE ROLE SYSADMIN;

CREATE OR REPLACE WAREHOUSE SHOPFLOW_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE DATABASE SHOPFLOW;
CREATE SCHEMA SHOPFLOW.RAW;        -- dane surowe i stage
CREATE SCHEMA SHOPFLOW.ANALYTICS;  -- dane przetworzone
CREATE SCHEMA SHOPFLOW.PIPELINE;   -- taski (graf tasków musi być w jednym schemacie)
CREATE SCHEMA SHOPFLOW.UTIL;       -- pomocnicze dane do generowania plików

USE WAREHOUSE SHOPFLOW_WH;
USE DATABASE SHOPFLOW;
```

Krok 0.2. Formaty plików i stage

```
-- Format do ładowania (COPY INTO tabela)
CREATE OR REPLACE FILE FORMAT SHOPFLOW.RAW.FF_CSV
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  TRIM_SPACE = TRUE
  NULL_IF = ('', 'NULL')
  EMPTY_FIELD_AS_NULL = TRUE
  COMPRESSION = AUTO;

-- Format do generowania plików (COPY INTO @stage)
CREATE OR REPLACE FILE FORMAT SHOPFLOW.RAW.FF_CSV_UNLOAD
  TYPE = CSV
  FIELD_DELIMITER = ','
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  COMPRESSION = NONE;

-- Wewnętrzny stage – "skrzynka odbiorcza" na pliki z systemu sprzedaży
CREATE OR REPLACE STAGE SHOPFLOW.RAW.LANDING_STAGE
  FILE_FORMAT = SHOPFLOW.RAW.FF_CSV
  COMMENT = 'Pliki CSV z zamowieniami eksportowane co noc z systemu sprzedazy';
```

Krok 0.3. Słownik produktów

```
CREATE OR REPLACE TABLE SHOPFLOW.ANALYTICS.DIM_PRODUCTS (
  product_id   INTEGER,
  sku          VARCHAR(30),
  product_name VARCHAR(100),
  category     VARCHAR(50),
  list_price   NUMBER(10,2)
);
```

```
INSERT INTO SHOPFLOW.ANALYTICS.DIM_PRODUCTS VALUES
  (1, 'SKU-LAPTOP-14',  'Laptop 14 cali',          'Elektronika', 3499.00),
  (2, 'SKU-MONITOR-27', 'Monitor 27 cali',         'Elektronika', 1299.00),
  (3, 'SKU-MOUSE',      'Mysz bezprzewodowa',      'Akcesoria',     89.00),
  (4, 'SKU-KEYBOARD',   'Klawiatura mechaniczna',  'Akcesoria',    349.00),
  (5, 'SKU-HEADSET',    'Sluchawki z mikrofonem',  'Audio',        249.00),
  (6, 'SKU-DOCK',       'Stacja dokujaca USB-C',   'Akcesoria',    599.00),
  (7, 'SKU-WEBCAM',     'Kamera internetowa 4K',   'Wideo',        449.00),
  (8, 'SKU-SSD-1TB',    'Dysk SSD 1TB',            'Pamieci',      399.00);
```

Krok 0.4. Generowanie 1000 zamówień z trzech dni

Rozkład: 400 zamówień z 01.11.2026, 350 z 02.11.2026 i 250 z 03.11.2026. Wartości (produkt, ilość, klient, kraj, status) są losowane, liczby wierszy są stałe – dzięki temu testy weryfikacyjne dają powtarzalne wyniki.

```
CREATE OR REPLACE TABLE SHOPFLOW.UTIL.ORDERS_SOURCE AS
WITH base AS (
  SELECT
    ROW_NUMBER() OVER (ORDER BY SEQ4()) AS rn,
    UNIFORM(1, 8, RANDOM())     AS product_no,
    UNIFORM(1, 5, RANDOM())     AS quantity,
    UNIFORM(0, 86399, RANDOM()) AS sec_of_day,
    UNIFORM(1, 300, RANDOM())   AS customer_no,
    UNIFORM(1, 5, RANDOM())     AS country_no,
    UNIFORM(1, 20, RANDOM())    AS status_no
  FROM TABLE(GENERATOR(ROWCOUNT => 1000))
),
dated AS (
  SELECT b.*,
         CASE WHEN rn <= 400 THEN '2026-11-01'::DATE
              WHEN rn <= 750 THEN '2026-11-02'::DATE
              ELSE '2026-11-03'::DATE END AS order_date
  FROM base b
)
SELECT
  d.order_date,
  'ORD-' || TO_CHAR(d.order_date, 'YYYYMMDD') || '-'
         || LPAD(d.rn::VARCHAR, 5, '0')                       AS order_id,
  DATEADD(second, d.sec_of_day, d.order_date::TIMESTAMP_NTZ) AS order_ts,
  'C' || LPAD(d.customer_no::VARCHAR, 4, '0')               AS customer_id,
  p.sku,
  d.quantity,
  p.list_price                                               AS unit_price,
  DECODE(d.country_no, 1, 'PL', 2, 'DE', 3, 'CZ', 4, 'SK', 'LT') AS country,
  CASE WHEN d.status_no = 1 THEN 'CANCELLED'
       WHEN d.status_no = 2 THEN 'RETURNED'
       ELSE 'COMPLETED' END                                  AS status
FROM dated d
JOIN SHOPFLOW.ANALYTICS.DIM_PRODUCTS p ON p.product_id = d.product_no;
```

Krok 0.5. Wygenerowanie plików CSV w stage

Dwa „nocne” pliki z 01.11 i 02.11 oraz plik `orders_2026_11_02_late.csv` – późna dosyłka 10 zamówień, w której są 2 błędne wiersze (ilość zapisana słownie i cena „N/A”). To typowa sytuacja z życia: system źródłowy czasem wysyła brudne dane.

```
-- Plik z 01.11.2026 (400 zamówień)
COPY INTO @SHOPFLOW.RAW.LANDING_STAGE/orders/orders_2026_11_01.csv
FROM (
  SELECT order_id, order_ts, customer_id, sku, quantity, unit_price, country, status
  FROM SHOPFLOW.UTIL.ORDERS_SOURCE
  WHERE order_date = '2026-11-01'
  ORDER BY order_ts
)
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV_UNLOAD')
HEADER = TRUE SINGLE = TRUE OVERWRITE = TRUE;
```

```
-- Plik z 02.11.2026 (350 zamówień)
COPY INTO @SHOPFLOW.RAW.LANDING_STAGE/orders/orders_2026_11_02.csv
FROM (
  SELECT order_id, order_ts, customer_id, sku, quantity, unit_price, country, status
  FROM SHOPFLOW.UTIL.ORDERS_SOURCE
  WHERE order_date = '2026-11-02'
  ORDER BY order_ts
)
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV_UNLOAD')
HEADER = TRUE SINGLE = TRUE OVERWRITE = TRUE;

-- Późna dosyłka z 02.11.2026: 10 wierszy, w tym 2 błędne (wiersz 3 i 7)
COPY INTO @SHOPFLOW.RAW.LANDING_STAGE/orders/orders_2026_11_02_late.csv
FROM (
  SELECT
    'ORD-20261102-L' || LPAD(rn::VARCHAR, 3, '0')                          AS order_id,
    DATEADD(minute, rn * 5, '2026-11-02 23:00:00'::TIMESTAMP_NTZ)::VARCHAR AS order_ts,
    'C' || LPAD((100 + rn)::VARCHAR, 4, '0')                               AS customer_id,
    'SKU-MOUSE'                                                            AS sku,
    IFF(rn = 3, 'dwa', '1')                                                AS quantity,
    IFF(rn = 7, 'N/A', '89.00')                                            AS unit_price,
    'PL'                                                                   AS country,
    'COMPLETED'                                                            AS status
  FROM (SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) AS rn
        FROM TABLE(GENERATOR(ROWCOUNT => 10)))
)
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV_UNLOAD')
HEADER = TRUE SINGLE = TRUE OVERWRITE = TRUE;
```

Krok 0.6. Test startowy

```
SELECT 'Tabela DIM_PRODUCTS ma 8 produktow' AS test,
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.DIM_PRODUCTS) = 8, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT 'Wygenerowano 1000 zamowien zrodlowych',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.UTIL.ORDERS_SOURCE) = 1000, 'PASS', 'FAIL')
UNION ALL
SELECT 'W stage sa 3 pliki CSV',
       IFF((SELECT COUNT(DISTINCT METADATA$FILENAME)
            FROM @SHOPFLOW.RAW.LANDING_STAGE/orders/
                 (FILE_FORMAT => 'SHOPFLOW.RAW.FF_CSV')) = 3, 'PASS', 'FAIL');
```

Oczekiwany wynik: trzy wiersze ze statusem PASS.

3. Moduł 1. Stage i COPY INTO – ładowanie plików

Cel modułu

Potrafsz przejrzeć zawartość stage, odpytać pliki bez ładowania ich do tabeli, załadować dane poleceniem COPY INTO, obsłużyć błędne wiersze i wyjaśnić, dlaczego ponowne COPY nie dubluje danych.

Krok 1.1. Co jest w stage?

```
USE ROLE SYSADMIN;
USE WAREHOUSE SHOPFLOW_WH;
USE SCHEMA SHOPFLOW.RAW;

LIST @LANDING_STAGE/orders/;
```

Oczekiwany wynik: 3 pliki: `orders_2026_11_01.csv`, `orders_2026_11_02.csv`, `orders_2026_11_02_late.csv` wraz z rozmiarem, sumą MD5 i datą modyfikacji.

Krok 1.2. Podgląd plików bez ładowania

Snowflake pozwala odpytać plik w stage jak tabelę. Kolumny są dostępne jako `$1`, `$2`… a metadane jako `METADATA$FILENAME` i `METADATA$FILE_ROW_NUMBER`. Tak sprawdzamy strukturę pliku, zanim zaprojektujemy tabelę.

```
SELECT METADATA$FILENAME        AS plik,
       METADATA$FILE_ROW_NUMBER AS nr_wiersza,
       $1 AS order_id, $2 AS order_ts, $3 AS customer_id, $4 AS sku,
       $5 AS quantity, $6 AS unit_price, $7 AS country, $8 AS status
FROM @LANDING_STAGE/orders/ (FILE_FORMAT => 'SHOPFLOW.RAW.FF_CSV')
LIMIT 10;

-- Ile wierszy ma każdy plik?
SELECT METADATA$FILENAME AS plik, COUNT(*) AS wierszy
FROM @LANDING_STAGE/orders/ (FILE_FORMAT => 'SHOPFLOW.RAW.FF_CSV')
GROUP BY 1 ORDER BY 1;
```

Oczekiwany wynik: 400, 350 i 10 wierszy.

Krok 1.3. Tabela surowa

```
CREATE OR REPLACE TABLE SHOPFLOW.RAW.ORDERS_RAW (
  order_id    VARCHAR(30),
  order_ts    TIMESTAMP_NTZ,
  customer_id VARCHAR(10),
  sku         VARCHAR(30),
  quantity    NUMBER(10,0),
  unit_price  NUMBER(10,2),
  country     VARCHAR(2),
  status      VARCHAR(20)
);
```

Krok 1.4. Ładowanie „nocnych” plików

Parametr `PATTERN` (wyrażenie regularne) wybiera tylko pliki dzienne. Plik `_late` nie pasuje do wzorca, bo po dacie nie ma od razu `.csv`.

```
COPY INTO SHOPFLOW.RAW.ORDERS_RAW
FROM @LANDING_STAGE/orders/
PATTERN = '.*orders_2026_11_0[12][.]csv'
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV');
```

Oczekiwany wynik: 2 wiersze ze statusem `LOADED`, `rows_loaded` = 400 i 350, `errors_seen` = 0.

Krok 1.5. Uruchom to samo COPY jeszcze raz

```
COPY INTO SHOPFLOW.RAW.ORDERS_RAW
FROM @LANDING_STAGE/orders/
PATTERN = '.*orders_2026_11_0[12][.]csv'
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV');

SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW;  -- nadal 750
```

Oczekiwany wynik: komunikat „Copy executed with 0 files processed". Snowflake przechowuje przez 64 dni metadane ładowania (nazwa pliku, ETag/MD5) dla każdej tabeli i nie ładuje drugi raz tego samego pliku. Wymuszenie ponownego ładowania to `FORCE = TRUE` – ze świadomością, że powstaną duplikaty.

Krok 1.6. Plik z błędami – najpierw walidacja, potem ładowanie

```
-- 1) Walidacja bez ładowania danych
COPY INTO SHOPFLOW.RAW.ORDERS_RAW
FROM @LANDING_STAGE/orders/
FILES = ('orders_2026_11_02_late.csv')
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV')
VALIDATION_MODE = RETURN_ERRORS;

-- 2) Domyślne zachowanie (ON_ERROR = ABORT_STATEMENT) - polecenie zakonczy sie bledem
COPY INTO SHOPFLOW.RAW.ORDERS_RAW
FROM @LANDING_STAGE/orders/
FILES = ('orders_2026_11_02_late.csv')
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV');

-- 3) Ładujemy poprawne wiersze, błędne pomijamy
COPY INTO SHOPFLOW.RAW.ORDERS_RAW
FROM @LANDING_STAGE/orders/
FILES = ('orders_2026_11_02_late.csv')
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV')
ON_ERROR = CONTINUE;
```

Oczekiwany wynik: walidacja zwraca 2 błędy (wiersz 3: `Numeric value 'dwa' is not recognized`, wiersz 7: `Numeric value 'N/A' is not recognized`). Krok 2 kończy się błędem i nic nie ładuje. Krok 3 zwraca status `PARTIALLY_LOADED`, `rows_loaded` = 8, `errors_seen` = 2.

Krok 1.7. Historia ładowania

```
SELECT file_name, status, row_count, row_parsed, error_count, first_error_message, last_load_time
FROM TABLE(SHOPFLOW.INFORMATION_SCHEMA.COPY_HISTORY(
       TABLE_NAME => 'SHOPFLOW.RAW.ORDERS_RAW',
       START_TIME => DATEADD('hour', -2, CURRENT_TIMESTAMP())))
ORDER BY last_load_time;
```

Weryfikacja modułu 1

```
SELECT '1.1 ORDERS_RAW zawiera 758 wierszy' AS test,
       IFF((SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW) = 758, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '1.2 Brak duplikatow ORDER_ID',
       IFF((SELECT COUNT(*) - COUNT(DISTINCT order_id) FROM SHOPFLOW.RAW.ORDERS_RAW) = 0, 'PASS',
'FAIL')
UNION ALL
SELECT '1.3 Z pliku late zaladowano 8 poprawnych wierszy',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW
            WHERE order_id LIKE 'ORD-20261102-L%') = 8, 'PASS', 'FAIL')
UNION ALL
SELECT '1.4 Brak wartosci NULL w ilosci i cenie',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW
            WHERE quantity IS NULL OR unit_price IS NULL) = 0, 'PASS', 'FAIL')
UNION ALL
```

```
SELECT '1.5 Historia COPY: 3 pliki, w tym 1 czesciowo zaladowany',
       IFF((SELECT COUNT(DISTINCT file_name) || '/' ||
                   COUNT_IF(status = 'Partially loaded')
            FROM TABLE(SHOPFLOW.INFORMATION_SCHEMA.COPY_HISTORY(
                   TABLE_NAME => 'SHOPFLOW.RAW.ORDERS_RAW',
                   START_TIME => DATEADD('hour', -2, CURRENT_TIMESTAMP())))) = '3/1',
           'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli test 1.1 pokazuje 750, nie załadowano pliku late (krok 1.6, punkt 3). Jeśli pokazuje 1508 lub więcej, ktoś użył `FORCE = TRUE` – wtedy uruchom ponownie krok 1.3 (CREATE OR REPLACE) i powtórz ładowanie.

4. Moduł 2. Stream i INSERT INTO – przetwarzanie przyrostowe

Cel modułu

Tworzysz stream na tabeli surowej, rozumie pojęcie offsetu i kolumny METADATA$, ładuje dane do warstwy analitycznej poleceniem INSERT INTO … SELECT ze streamu i wie, kiedy stream jest „konsumowany”.

Krok 2.1. Tabela analityczna

```
USE SCHEMA SHOPFLOW.ANALYTICS;

CREATE OR REPLACE TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN (
  order_id     VARCHAR(30),
  order_date   DATE,
  order_ts     TIMESTAMP_NTZ,
  customer_id  VARCHAR(10),
  sku          VARCHAR(30),
  product_name VARCHAR(100),
  category     VARCHAR(50),
  quantity     NUMBER(10,0),
  unit_price   NUMBER(10,2),
  net_amount   NUMBER(12,2),
  country      VARCHAR(2),
  status       VARCHAR(20),
  processed_at TIMESTAMP_LTZ
);
```

Krok 2.2. Stream na tabeli surowej

Tabela surowa przyjmuje tylko nowe wiersze, więc używamy streamu **append-only** – jest tańszy i prostszy niż standardowy. Opcja `SHOW_INITIAL_ROWS = TRUE` sprawia, że przy pierwszym odczycie stream zwróci także 758 wierszy, które już są w tabeli. Dzięki temu ładowanie początkowe i przyrostowe to to samo zapytanie.

```
CREATE OR REPLACE STREAM SHOPFLOW.RAW.ORDERS_RAW_STREAM
  ON TABLE SHOPFLOW.RAW.ORDERS_RAW
  APPEND_ONLY = TRUE
  SHOW_INITIAL_ROWS = TRUE;

SHOW STREAMS IN SCHEMA SHOPFLOW.RAW;
```

Krok 2.3. Odczyt streamu (SELECT nie konsumuje)

```
SELECT METADATA$ACTION, METADATA$ISUPDATE, METADATA$ROW_ID, *
FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM
LIMIT 5;

SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM;  -- 758
SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM;  -- nadal 758
```

Oczekiwany wynik: `METADATA$ACTION` = INSERT, `METADATA$ISUPDATE` = FALSE; dwukrotny COUNT daje 758. Zwykły SELECT nie przesuwa offsetu streamu.

Krok 2.4. INSERT INTO ze streamu – ładowanie początkowe

To zapytanie jest sercem pipeline – w module 3 przeniesiemy je bez zmian do taska.

```
INSERT INTO SHOPFLOW.ANALYTICS.ORDERS_CLEAN
  (order_id, order_date, order_ts, customer_id, sku, product_name, category,
   quantity, unit_price, net_amount, country, status, processed_at)
SELECT s.order_id,
       s.order_ts::DATE,
       s.order_ts,
       s.customer_id,
```

```
       s.sku,
       p.product_name,
       p.category,
       s.quantity,
       s.unit_price,
       s.quantity * s.unit_price,
       s.country,
       s.status,
       CURRENT_TIMESTAMP()
FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM s
LEFT JOIN SHOPFLOW.ANALYTICS.DIM_PRODUCTS p ON p.sku = s.sku;

SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM;  -- 0
SELECT SYSTEM$STREAM_HAS_DATA('SHOPFLOW.RAW.ORDERS_RAW_STREAM');  -- FALSE
```

Oczekiwany wynik: wstawiono 758 wierszy, a stream jest pusty. Instrukcja DML, która czyta stream, przesuwa jego offset po zatwierdzeniu transakcji.

Krok 2.5. Nowe dane – zamówienia telefoniczne wpisane ręcznie

Dział obsługi klienta dopisuje 2 zamówienia przyjęte przez telefon bezpośrednio do tabeli surowej.

```
INSERT INTO SHOPFLOW.RAW.ORDERS_RAW
  (order_id, order_ts, customer_id, sku, quantity, unit_price, country, status)
VALUES
  ('ORD-20261103-M001', '2026-11-03 09:15:00', 'C0042', 'SKU-LAPTOP-14', 1, 3499.00, 'PL',
'COMPLETED'),
  ('ORD-20261103-M002', '2026-11-03 09:40:00', 'C0117', 'SKU-HEADSET',   2,  249.00, 'DE',
'COMPLETED');

SELECT METADATA$ACTION, order_id, sku, quantity
FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM;   -- dokładnie 2 nowe wiersze
```

Teraz uruchom **ponownie zapytanie z kroku 2.4** (bez zmian). Wstawi tylko 2 nowe wiersze – to jest przetwarzanie przyrostowe.

Weryfikacja modułu 2

```
SELECT '2.1 ORDERS_RAW zawiera 760 wierszy' AS test,
       IFF((SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW) = 760, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '2.2 ORDERS_CLEAN zawiera 760 wierszy',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN) = 760, 'PASS', 'FAIL')
UNION ALL
SELECT '2.3 Stream jest pusty (wszystko przetworzone)',
       IFF(NOT SYSTEM$STREAM_HAS_DATA('SHOPFLOW.RAW.ORDERS_RAW_STREAM'), 'PASS', 'FAIL')
UNION ALL
SELECT '2.4 Kazde zamowienie z RAW jest w CLEAN dokladnie raz',
       IFF((SELECT COUNT(*)
            FROM SHOPFLOW.RAW.ORDERS_RAW r
            LEFT JOIN (SELECT order_id, COUNT(*) AS cnt
                       FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN GROUP BY order_id) c
                   ON c.order_id = r.order_id
            WHERE COALESCE(c.cnt, 0) <> 1) = 0, 'PASS', 'FAIL')
UNION ALL
SELECT '2.5 Wartosc netto M002 = 498.00 i produkt wzbogacony',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN
            WHERE order_id = 'ORD-20261103-M002'
              AND net_amount = 498.00
              AND product_name = 'Sluchawki z mikrofonem') = 1, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli CLEAN ma 762 lub więcej wierszy, zapytanie z kroku 2.4 uruchomiono ze zwykłej tabeli zamiast ze streamu – usuń duplikaty poleceniem `CREATE OR REPLACE TABLE … AS SELECT DISTINCT …` albo wróć do kroku 2.1 i 2.2.

5. Moduł 3. Task – automatyzacja pipeline

Cel modułu

Budujesz graf dwóch tasków: task główny co minutę ładuje nowe pliki (COPY INTO), a task zależny przetwarza nowe wiersze ze streamu (INSERT INTO), ale tylko wtedy, gdy stream ma dane. Potrafisz uruchomić task ręcznie, monitorować historię i zawiesić taski.

Krok 3.1. Task główny – ładowanie plików

Dzięki idempotentności COPY (moduł 1) task może bezpiecznie uruchamiać się co minutę na całym katalogu – załaduje tylko pliki, których jeszcze nie widział.

```
USE SCHEMA SHOPFLOW.PIPELINE;

CREATE OR REPLACE TASK SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS
  WAREHOUSE = SHOPFLOW_WH
  SCHEDULE = '1 MINUTE'
  COMMENT = 'Laduje nowe pliki z zamowieniami ze stage do RAW'
AS
  COPY INTO SHOPFLOW.RAW.ORDERS_RAW
  FROM @SHOPFLOW.RAW.LANDING_STAGE/orders/
  PATTERN = '.*orders_.*[.]csv'
  FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV')
  ON_ERROR = CONTINUE;
```

Krok 3.2. Task zależny – transformacja ze streamu

Klauzula `AFTER` wpina task w graf, a `WHEN SYSTEM$STREAM_HAS_DATA(…)` sprawia, że jeśli nie ma nowych danych, task zostaje pominięty bez uruchamiania warehouse. Treść taska to dokładnie zapytanie z kroku 2.4.

```
CREATE OR REPLACE TASK SHOPFLOW.PIPELINE.TSK_TRANSFORM_ORDERS
  WAREHOUSE = SHOPFLOW_WH
  AFTER SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS
  WHEN SYSTEM$STREAM_HAS_DATA('SHOPFLOW.RAW.ORDERS_RAW_STREAM')
  COMMENT = 'Przenosi nowe zamowienia z RAW do ANALYTICS'
AS
  INSERT INTO SHOPFLOW.ANALYTICS.ORDERS_CLEAN
    (order_id, order_date, order_ts, customer_id, sku, product_name, category,
     quantity, unit_price, net_amount, country, status, processed_at)
  SELECT s.order_id, s.order_ts::DATE, s.order_ts, s.customer_id, s.sku,
         p.product_name, p.category, s.quantity, s.unit_price,
         s.quantity * s.unit_price, s.country, s.status, CURRENT_TIMESTAMP()
  FROM SHOPFLOW.RAW.ORDERS_RAW_STREAM s
  LEFT JOIN SHOPFLOW.ANALYTICS.DIM_PRODUCTS p ON p.sku = s.sku;
```

Krok 3.3. Włączenie grafu

Nowe taski są zawieszone (`suspended`). Kolejność ma znaczenie: najpierw wznawiamy taski zależne, na końcu task główny. Alternatywnie: `SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS');`.

```
ALTER TASK SHOPFLOW.PIPELINE.TSK_TRANSFORM_ORDERS RESUME;
ALTER TASK SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS RESUME;

SHOW TASKS IN SCHEMA SHOPFLOW.PIPELINE;   -- kolumna state = started
```

Krok 3.4. Symulacja: dotarł plik z 03.11.2026

System sprzedaży eksportuje 250 zamówień z kolejnego dnia. Nie ładujemy ich ręcznie – zrobi to pipeline.

```
COPY INTO @SHOPFLOW.RAW.LANDING_STAGE/orders/orders_2026_11_03.csv
FROM (
  SELECT order_id, order_ts, customer_id, sku, quantity, unit_price, country, status
```

```
  FROM SHOPFLOW.UTIL.ORDERS_SOURCE
  WHERE order_date = '2026-11-03'
  ORDER BY order_ts
)
FILE_FORMAT = (FORMAT_NAME = 'SHOPFLOW.RAW.FF_CSV_UNLOAD')
HEADER = TRUE SINGLE = TRUE OVERWRITE = TRUE;

LIST @SHOPFLOW.RAW.LANDING_STAGE/orders/;   -- teraz 4 pliki
```

Krok 3.5. Uruchomienie bez czekania na harmonogram

Task uruchomi się sam w ciągu minuty. Na warsztacie przyspieszamy to poleceniem `EXECUTE TASK` na tasku głównym – wykona się cały graf.

```
EXECUTE TASK SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS;

-- Odczekaj ok. 30-60 sekund i sprawdź historię
SELECT name, state, scheduled_time, completed_time, error_message
FROM TABLE(SHOPFLOW.INFORMATION_SCHEMA.TASK_HISTORY(
       SCHEDULED_TIME_RANGE_START => DATEADD('minute', -30, CURRENT_TIMESTAMP()),
       RESULT_LIMIT => 50))
ORDER BY scheduled_time DESC;
```

Oczekiwany wynik: `TSK_LOAD_ORDERS` i `TSK_TRANSFORM_ORDERS` ze stanem `SUCCEEDED`. Kolejne cykliczne przebiegi taska głównego kończą się szybko (0 nowych plików), a task zależny ma stan `SKIPPED` – warunek WHEN nie był spełniony. Stan `SCHEDULED` oznacza, że przebieg jeszcze się nie zakończył – odczekaj i powtórz zapytanie.

Krok 3.6. Zawieszenie tasków

Po weryfikacji zawieś task główny, aby nie zużywał kredytów co minutę. Zawieszenie taska głównego zatrzymuje cały graf.

```
ALTER TASK SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS SUSPEND;
```

> **Wskazówka dla trenera**
>
> Zamiast `WAREHOUSE = …` można użyć tasków serverless (`USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'`) – Snowflake sam dobiera zasoby, rozliczenie jest za faktyczne użycie.
>
> W produkcji do ładowania plików częściej używa się Snowpipe (ładowanie zdarzeniowe). Task z COPY jest dobrym rozwiązaniem przy wsadach co kilka minut lub godzin.
>
> Wszystkie taski w jednym grafie muszą należeć do tego samego właściciela i znajdować się w tym samym schemacie – dlatego wydzieliliśmy schemat PIPELINE.

Weryfikacja modułu 3

```
SELECT '3.1 ORDERS_RAW zawiera 1010 wierszy' AS test,
       IFF((SELECT COUNT(*) FROM SHOPFLOW.RAW.ORDERS_RAW) = 1010, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '3.2 ORDERS_CLEAN zawiera 1010 wierszy',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN) = 1010, 'PASS', 'FAIL')
UNION ALL
SELECT '3.3 Zamowienia z 03.11 w CLEAN: 252 (250 z pliku + 2 reczne)',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN
            WHERE order_date = '2026-11-03') = 252, 'PASS', 'FAIL')
UNION ALL
SELECT '3.4 Oba taski maja udane uruchomienie (SUCCEEDED)',
       IFF((SELECT COUNT(DISTINCT name)
            FROM TABLE(SHOPFLOW.INFORMATION_SCHEMA.TASK_HISTORY(
                   SCHEDULED_TIME_RANGE_START => DATEADD('hour', -1, CURRENT_TIMESTAMP()),
                   RESULT_LIMIT => 1000))
            WHERE state = 'SUCCEEDED'
              AND name IN ('TSK_LOAD_ORDERS', 'TSK_TRANSFORM_ORDERS')) = 2, 'PASS', 'FAIL')
```

```
UNION ALL
SELECT '3.5 Plik orders_2026_11_03.csv zaladowal 250 wierszy',
       IFF((SELECT SUM(row_count)
            FROM TABLE(SHOPFLOW.INFORMATION_SCHEMA.COPY_HISTORY(
                   TABLE_NAME => 'SHOPFLOW.RAW.ORDERS_RAW',
                   START_TIME => DATEADD('hour', -2, CURRENT_TIMESTAMP())))
            WHERE file_name ILIKE '%orders_2026_11_03.csv') = 250, 'PASS', 'FAIL');

-- Kontrola stanu tasków (oczekiwane: TSK_LOAD_ORDERS = suspended)
SHOW TASKS IN SCHEMA SHOPFLOW.PIPELINE;
SELECT "name", "state" FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()));
```

Oczekiwany wynik: 5 × PASS oraz `TSK_LOAD_ORDERS` w stanie `suspended`. Jeśli 3.4 daje FAIL, sprawdź kolumnę `error_message` w historii. Najczęstsza przyczyna to brak uprawnienia `EXECUTE TASK` (krok 0.1) albo task zależny nie został wznowiony przed taskiem głównym.

6. Moduł 4. Time Travel – odtwarzanie po awarii

Cel modułu

Odczytujesz historyczny stan tabeli (AT/BEFORE), odtwarza dane po błędnym UPDATE za pomocą CLONE i SWAP oraz przywraca usuniętą tabelę poleceniem UNDROP.

Krok 4.1. Okres retencji i stan przed incydentem

```
USE SCHEMA SHOPFLOW.ANALYTICS;

SHOW PARAMETERS LIKE 'DATA_RETENTION_TIME_IN_DAYS' IN TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN;

-- Zapisujemy "zdjęcie" stanu do zmiennych sesji
SET rows_before      = (SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN);
SET cancelled_before = (SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN WHERE status =
'CANCELLED');

SELECT $rows_before AS wierszy, $cancelled_before AS anulowanych;

SELECT status, COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN GROUP BY 1 ORDER BY 1;
```

Oczekiwany wynik: retencja = 1 dzień (domyślnie; w edycji Enterprise można ustawić do 90 dni). Około 5% zamówień ma status CANCELLED i 5% RETURNED – dokładne liczby są losowe, dlatego zapisujemy je w zmiennych.

Krok 4.2. Incydent 1: UPDATE bez WHERE

Klient zadzwonił, że rezygnuje z zamówienia ORD-20261103-M002. Analityk w pośpiechu wykonuje:

```
UPDATE SHOPFLOW.ANALYTICS.ORDERS_CLEAN
SET status = 'CANCELLED';            -- zapomniał o WHERE!

SET bad_update_qid = LAST_QUERY_ID();  -- zapamiętujemy ID błędnego zapytania

SELECT status, COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN GROUP BY 1;  -- 1010 x CANCELLED
```

Jeśli ID zapytania nie zostało zapisane, znajdziesz je w Snowsight: **Monitoring → Query History**, albo zapytaniem do `TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())`.

Krok 4.3. Podróż w czasie – jak wyglądała tabela przed błędem?

```
-- Stan tuż przed wskazanym zapytaniem
SELECT status, COUNT(*)
FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN BEFORE (STATEMENT => $bad_update_qid)
GROUP BY 1 ORDER BY 1;

-- Inne warianty składni (do pokazania):
-- stan sprzed N sekund
SELECT COUNT_IF(status = 'CANCELLED')
FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN AT (OFFSET => -120);
-- stan na konkretny moment
-- SELECT * FROM ORDERS_CLEAN AT (TIMESTAMP => '2026-11-05 11:30:00'::TIMESTAMP_LTZ);
```

Oczekiwany wynik: zapytanie BEFORE pokazuje pierwotny rozkład statusów (taki jak w kroku 4.1). Zapytanie z OFFSET może zwrócić błąd, jeśli tabela jest młodsza niż podany czas, albo stan już po UPDATE, jeśli od incydentu minęło więcej niż 2 minuty – to dobry moment, by omówić, dlaczego `STATEMENT` jest najdokładniejszy.

Krok 4.4. Odtworzenie: CLONE z przeszłości + SWAP

Klon zero-copy tworzy się natychmiast i nie kopiuje danych fizycznie. SWAP atomowo zamienia nazwy tabel, więc raporty i task wskazujące na ORDERS_CLEAN od razu widzą poprawne dane.

```
CREATE OR REPLACE TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN_FIX
  CLONE SHOPFLOW.ANALYTICS.ORDERS_CLEAN BEFORE (STATEMENT => $bad_update_qid);

ALTER TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN SWAP WITH SHOPFLOW.ANALYTICS.ORDERS_CLEAN_FIX;

-- ORDERS_CLEAN_FIX zawiera teraz uszkodzone dane - usuwamy ją
DROP TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN_FIX;

-- Wykonujemy poprawną operację, którą analityk chciał zrobić
UPDATE SHOPFLOW.ANALYTICS.ORDERS_CLEAN
SET status = 'CANCELLED'
WHERE order_id = 'ORD-20261103-M002';
```

Krok 4.5. Incydent 2: przypadkowy DROP TABLE

```
DROP TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN;

SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN;  -- błąd: tabela nie istnieje

SHOW TABLES HISTORY LIKE 'ORDERS_CLEAN%' IN SCHEMA SHOPFLOW.ANALYTICS;  -- kolumna dropped_on

UNDROP TABLE SHOPFLOW.ANALYTICS.ORDERS_CLEAN;

SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN;  -- 1010
```

> **Wskazówka**
>
> Po upływie okresu retencji dane trafiają na 7 dni do Fail-safe – tam może je odzyskać tylko wsparcie Snowflake. Fail-safe nie jest narzędziem dla użytkownika.
>
> Retencja kosztuje: przechowywane są wszystkie zmienione mikropartycje. Dla tabel tymczasowych i przejściowych (TRANSIENT) można ustawić retencję 0 dni.
>
> UNDROP działa także dla schematów i baz danych (UNDROP SCHEMA, UNDROP DATABASE).

Weryfikacja modułu 4

```
SELECT '4.1 Liczba wierszy taka jak przed incydentem' AS test,
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN) = $rows_before, 'PASS', 'FAIL') AS
wynik
UNION ALL
SELECT '4.2 Anulowanych o dokladnie 1 wiecej niz przed incydentem',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN
            WHERE status = 'CANCELLED') = $cancelled_before + 1, 'PASS', 'FAIL')
UNION ALL
SELECT '4.3 M002 anulowane, M001 nadal zrealizowane',
       IFF((SELECT LISTAGG(status, ',') WITHIN GROUP (ORDER BY order_id)
            FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN
            WHERE order_id IN ('ORD-20261103-M001', 'ORD-20261103-M002'))
           = 'COMPLETED,CANCELLED', 'PASS', 'FAIL')
UNION ALL
SELECT '4.4 W tabeli wystepuja 3 rozne statusy',
       IFF((SELECT COUNT(DISTINCT status) FROM SHOPFLOW.ANALYTICS.ORDERS_CLEAN) = 3, 'PASS', 'FAIL')
UNION ALL
SELECT '4.5 Tabela robocza ORDERS_CLEAN_FIX usunieta',
       IFF((SELECT COUNT(*) FROM SHOPFLOW.INFORMATION_SCHEMA.TABLES
            WHERE table_schema = 'ANALYTICS' AND table_name = 'ORDERS_CLEAN_FIX') = 0, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Testy 4.1 i 4.2 korzystają ze zmiennych sesji z kroku 4.1. Jeśli arkusz został w międzyczasie zamknięty, zmienne trzeba odtworzyć z przeszłości: `SET cancelled_before = (SELECT COUNT(*) FROM ORDERS_CLEAN BEFORE (STATEMENT => '<query_id>') WHERE status = 'CANCELLED');`.

7. Podsumowanie i sprzątanie

7.1. Co zbudowaliśmy

Funkcja	Czego się nauczyliśmy
Stage	Przechowywanie plików w Snowflake, odpytywanie plików przez $1…$n i METADATA$FILENAME bez ładowania
COPY INTO	Ładowanie z PATTERN/FILES, idempotentność (64 dni metadanych), VALIDATION_MODE, ON_ERROR, COPY_HISTORY; a także unload do stage
INSERT INTO	Wstawianie wartości i INSERT … SELECT z transformacją i wzbogaceniem danych
Stream	Śledzenie zmian (CDC), offset, APPEND_ONLY, SHOW_INITIAL_ROWS, konsumpcja tylko przez DML
Task	Harmonogram, graf AFTER, warunek WHEN SYSTEM$STREAM_HAS_DATA, EXECUTE TASK, TASK_HISTORY, kolejność RESUME
Time Travel	AT/BEFORE (OFFSET, TIMESTAMP, STATEMENT), CLONE z przeszłości, SWAP, UNDROP, retencja i Fail-safe

7.2. Pytania kontrolne

1. Dlaczego task z COPY może bezpiecznie uruchamiać się co minutę na tym samym katalogu?
2. Co się stanie ze streamem, jeśli INSERT ze streamu zakończy się błędem w połowie transakcji?
3. Kiedy zamiast INSERT ze streamu należy użyć MERGE?
4. Jak długo możemy cofnąć skutki błędu w tabeli przy domyślnych ustawieniach i co dzieje się później?

7.3. Sprzątanie

```
USE ROLE SYSADMIN;
ALTER TASK IF EXISTS SHOPFLOW.PIPELINE.TSK_LOAD_ORDERS SUSPEND;
DROP DATABASE IF EXISTS SHOPFLOW;
DROP WAREHOUSE IF EXISTS SHOPFLOW_WH;

-- Bonus: baza też podlega Time Travel
-- UNDROP DATABASE SHOPFLOW;
```